

Video 1.5: Container Attack Surface Mapping

Duration: 7:00

Format: PPT (25%) / Terminal (35%) / Browser (25%) / Code Editor (15%)

Resolution: 1920x1080 @ 30fps

Prerequisites: Videos 1.1-1.4

SCRIPT BEGIN

[0:00-0:50] — The Attack Surface Framework (PPT Slide)

SLIDE: Title: "Container Attack Surface — Complete Map"

Diagram showing concentric rings from inside out:

- Inner ring: **Application Layer** (code vulnerabilities, dependencies)
- Second ring: **Container Image Layer** (base image, installed packages, secrets in layers)
- Third ring: **Runtime Layer** (Docker daemon, container config, capabilities)
- Fourth ring: **Orchestration Layer** (Kubernetes, Docker Swarm, API servers)
- Outer ring: **Host/Kernel Layer** (kernel, host OS, shared resources)

NARRATION:

"To defend containers effectively, you need a complete picture of the attack surface. I break it into five layers, from inside out. The application layer — your code and its dependencies. The container image layer — the base image, installed packages, and anything baked into the image. The runtime layer — how the container is configured, what privileges it has, the daemon itself. The orchestration layer — if you're using Kubernetes or Docker Swarm, that's a massive additional surface. And the host/kernel layer — which we've been discussing. Each layer has distinct attack vectors, and an adversary can move between layers. Let's examine each one."

[0:50-2:00] — Application Layer Attacks (Terminal + Browser)

NARRATION:

"Starting from the inside — application-level vulnerabilities."

TERMINAL:

```
# Pull a vulnerable application image for demonstration
$ docker run -d --name vuln-app -p 8080:8080 \
  vulnerables/web-dvwa
```

"I'm running DVWA — Damn Vulnerable Web Application — in a container. This simulates a real-world scenario: your application has a vulnerability."

```
# Show that application-level access leads to container access
# After exploiting a web vulnerability (e.g., command injection):
$ docker exec vuln-app id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

“If an attacker exploits a web vulnerability — SQL injection, command injection, deserialization — they get a shell inside the container. They’re running as the web application user. From here, every subsequent attack builds on container misconfiguration.”

BROWSER: Navigate to `http://localhost:8080` showing DVWA interface

“Here’s the DVWA interface. In a real attack, the attacker would exploit one of these vulnerabilities to get command execution inside the container. Then they’d pivot to exploiting container misconfiguration.”

```
# Clean up
$ docker rm -f vuln-app
```

[2:00-3:15] — Image Layer Attacks (Terminal)

NARRATION:

“The container image itself is a rich attack surface.”

COMMAND + EXPLANATION:

```
# Scan an image for known vulnerabilities
$ docker run --rm aquasec/trivy image nginx:latest
```

“Using Trivy — an open-source vulnerability scanner — we can scan any container image for known CVEs. Watch the output.”

```
# Sample output (abbreviated):
# nginx:latest (debian 12.4)
# Total: 143 (UNKNOWN: 0, LOW: 89, MEDIUM: 38, HIGH: 14, CRITICAL: 2)
#
#
```

Library	Vulnerability	Severity	Fixed Version
libssl3	CVE-2024-XXXX	CRITICAL	3.0.13-1
libcurl4	CVE-2024-XXXX	HIGH	7.88.1-10+deb12

```
#
```

“143 vulnerabilities in the official nginx image. 2 critical, 14 high. This is typical — base images carry significant vulnerability debt. And every package you install adds more.”

```
# Compare with a minimal image
$ docker run --rm aquasec/trivy image nginx:alpine
```

“Compare with the Alpine-based nginx — significantly fewer vulnerabilities because Alpine uses musl libc and has a smaller package set. Choosing minimal base images is your first line of defense.”

```
# Check for secrets accidentally baked into images
$ docker run --rm aquasec/trivy fs --scanners secret /path/to/dockerfile/context
```

“Trivy can also scan for secrets — API keys, passwords, private keys — accidentally baked into images. This is more common than you’d think.”

[3:15-4:30] — Runtime Layer Attacks (Terminal + Code Editor)

NARRATION:

“The runtime layer is where misconfigurations become exploitable.”

CODE EDITOR — Common dangerous `docker run` flags:

```
# ❌ DANGEROUS: Privileged mode – gives ALL capabilities + device access
docker run --privileged myapp

# ❌ DANGEROUS: Host PID namespace – can see all host processes
docker run --pid=host myapp

# ❌ DANGEROUS: Host network – bypasses network isolation
docker run --net=host myapp

# ❌ DANGEROUS: Docker socket mount – gives container root access to host
docker run -v /var/run/docker.sock:/var/run/docker.sock myapp

# ❌ DANGEROUS: Host filesystem mount
docker run -v /:/host myapp

# ❌ DANGEROUS: SYS_ADMIN capability – enough for container escape
docker run --cap-add=SYS_ADMIN myapp

# ✅ SECURE: Minimal privileges
docker run \
  --cap-drop=ALL \
  --security-opt=no-new-privileges \
  --read-only \
  --tmpfs /tmp:rw,noexec,nosuid,size=64m \
  --user 1000:1000 \
  --pids-limit=100 \
  --memory=256m \
  --cpus=0.5 \
  myapp
```

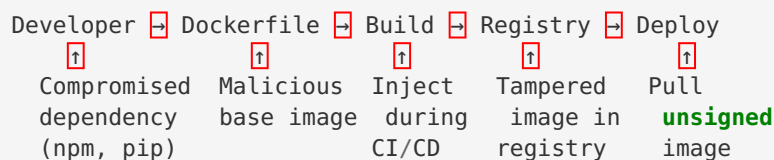
NARRATION:

“Here’s a gallery of dangerous docker run flags. Privileged mode — we’ll dissect this in Module 2. Host PID namespace — the container can see and interact with every process on the host. Host network — network isolation is completely removed. Docker socket mount — the container can control Docker and therefore the host. Host filesystem mount — self-explanatory. And SYS_ADMIN capability alone is often enough for container escape. At the bottom, the secure version — all capabilities dropped, no new privileges, read-only filesystem, non-root user, and resource limits. Every one of these flags is a checklist item in your container security audit.”

[4:30-5:30] — Supply Chain Attacks (PPT + Browser)

SLIDE: Title: “Container Supply Chain Attack Vectors”

Attack flow diagram:

**NARRATION:**

“Container supply chain attacks target every stage of the image lifecycle. A compromised npm or pip package in your Dockerfile pulls in malicious code. A malicious base image — there have been real cases of typosquatted official images on Docker Hub. Injection during CI/CD — if your build pipeline is compromised, every image it produces is compromised. Registry tampering — if the registry isn’t secured, images can be replaced. And pulling unsigned images — without content trust, you’re trusting the network blindly.”

BROWSER: Navigate to <https://hub.docker.com/>

“Docker Hub hosts millions of images. The official images — marked with the blue badge — are vetted. But anyone can push an image called `nginx` with a typo, or `mysql-server` — a name that looks official but isn’t. Without content trust and registry scanning, these make it into production.”

[5:30-6:20] — Orchestration Layer Preview (PPT Slide)

SLIDE: Title: “Orchestration Attack Surface (Kubernetes Preview)”

Component	Risk	Impact
API Server	Unauthenticated access	Full cluster control
etcd	Unencrypted data	Secret extraction
Kubelet	Anonymous auth	Node compromise
Service Accounts	Over-permissioned	Lateral movement
Pod Security	No policies	Privileged pods
RBAC	Misconfigured	Privilege escalation
Network Policies	Missing	No pod isolation
Admission Controllers	Disabled	Security bypass

NARRATION:

“If you’re running Kubernetes, you’ve added an enormous attack surface. The API server, etcd, kubelet, service accounts, RBAC, network policies, admission controllers — each one is a potential entry point. We won’t cover Kubernetes security in depth in this course, but be aware: every layer of orchestration you add is another layer of attack surface. The same principles apply — minimize access, enforce least privilege, monitor everything.”

[6:20-7:00] — Module Recap and What's Next (PPT Slide)

SLIDE: Title: "Module 1 — Complete"

What you now understand:

-  Container isolation is built on namespaces, cgroups, and layered filesystems
-  Containers share the host kernel — fundamentally different from VMs
-  300+ system calls are exposed by default
-  Docker daemon misconfiguration = host compromise
-  The attack surface spans five layers — application to host

What's next:

-  Module 2: Privileged Containers & Attack Vectors
-  You'll exploit the weaknesses we discussed today

NARRATION:

"That completes Module 1. You now have the security-focused mental model of container architecture that you'll carry through the rest of this course. You understand what containers actually are at the kernel level, where the isolation boundaries are, and where they fail. You know the Docker daemon is a high-value target and how to harden it. And you have a complete map of the attack surface. In Module 2, we're going to take everything we've discussed and weaponize it. We'll look at privileged containers — the most dangerous misconfiguration in container security — and demonstrate real container escapes. Don't forget to complete the module assessment before moving on. I'll see you in Module 2."

SCRIPT END

Post-Production Notes

- The attack surface concentric ring diagram should be animated — rings appearing from inside out
- Trivy scan should show real output — pre-run against nginx:latest before recording
- The "dangerous flags" code editor section should use red highlighting for  lines and green for 
- DVWA should be pre-configured and loaded — do not show setup screens
- Docker Hub browser segment should be brief — pre-navigate to search results showing unofficial images
- End with a 3-second module completion animation before transition
- Consider adding a "Module 1 Complete" achievement badge animation